

LLM as a Judge

Como um segundo modelo avalia, regra por regra,
as respostas de um agente de IA

Estudo de caso: atendimento de call center da NetVia Internet
Material de apoio — Eval na prática

1. O que é LLM as a Judge?

"LLM as a Judge" é uma técnica de avaliação (eval) em que um modelo de linguagem é usado para julgar a saída de outro sistema de IA — em vez de depender só de revisão humana ou de métricas exatas (como comparar string a string). O juiz recebe o contexto, um critério claro (regras, rubrica ou gabarito) e a resposta a avaliar, e devolve um veredito estruturado.

Por que não avaliar "na mão" ou com métricas simples?

- Respostas de LLMs são abertas e variam em texto, mas o que importa é o *conteúdo*: seguir ou não uma regra de negócio.
- Revisão 100% humana não escala para milhares de interações; o juiz permite avaliação contínua e automática.
- Uma resposta pode ser educada e bem escrita e, ainda assim, violar uma regra — só um avaliador que entende o significado percebe isso.

Como isso funciona nesta demo

- O **atendente** (agent.py) é o sistema avaliado: responde ao cliente usando os dados do sistema e as regras de negócio.
- O **juiz** (judge.py) é um segundo modelo, chamado logo depois, com **temperatura 0** (respostas mais determinísticas) e um schema de saída fixo.
- Para cada regra de negócio (R1 a R5, definidas em rules.py), o juiz classifica: **cumprida**, **violada** ou **não se aplica** — com uma justificativa curta.
- A **nota (0 a 10) e o veredito final são calculados em Python**, a partir dessas classificações — o modelo nunca "inventa" a nota, o que torna o resultado auditável e reproduzível.

Ponto-chave: separar "julgar cada regra" (tarefa do LLM) de "calcular a nota" (tarefa determinística em código) reduz a variabilidade e facilita auditar por que um atendimento foi aprovado ou reprovado.

2. Como implementar: o juiz em código

Três peças fazem o LLM as a Judge funcionar neste projeto: o gabarito (regras de negócio), o prompt do juiz e o cálculo determinístico da nota.

O gabarito — rules.py

```
BUSINESS_RULES = [
    {
        "id": "R1",
        "titulo": "Cumprimento e protocolo",
        "texto": (
            "Na primeira resposta do atendimento, cumprimentar o cliente pelo nome "
            "e informar o número de protocolo."
        ),
    },
    {
        "id": "R2",
        "titulo": "Consultar o status da região",
        "texto": (
            "Antes de orientar qualquer procedimento técnico, considerar o status "
            "da região informado pelo sistema."
        ),
    },
    # ... R3, R4 e R5 seguem o mesmo formato
]
```

As regras de negócio são a única fonte da verdade: tanto o atendente quanto o juiz leem esta mesma lista.

O prompt do juiz — judge.py

```
_SYSTEM = f"""Você é um avaliador rigoroso e imparcial (LLM as a Judge) de atendimentos de call center da {EMPRESA}. Sua tarefa é verificar se a ÚLTIMA resposta do atendente respeita as regras de negócio. Não avalie simpatia nem estilo de escrita: uma resposta pode ser educada e convincente e ainda assim violar uma regra.
```

```
Para cada regra, responda com um destes status:
- cumprida: a resposta segue a regra.
- violada: a resposta contraria a regra.
- nao_aplicavel: a regra não se aplica a esta resposta.
```

```
Baseie-se apenas nos dados do sistema, na conversa e na resposta avaliada."""
```

Note o pedido explícito para ignorar estilo/simpatia e focar só no cumprimento da regra.

A nota é calculada em Python, não pelo modelo — judge.py

```
cumpridas = sum(r.status == "cumprida" for r in regras)
violadas = [r.id for r in regras if r.status == "violada"]
avaliadas = cumpridas + len(violadas)
nota = round(10 * cumpridas / avaliadas, 1) if avaliadas else 10.0

return Avaliacao(
    regras=regras,
    resumo=parecer.resumo,
    nota=nota,
    aprovada=not violadas,
    violadas=violadas,
)
```

nota = 10 × (regras cumpridas) / (regras cumpridas + violadas). Regras "não se aplica" ficam fora da conta.

3. Simulação: quando o juiz acerta e quando pega o erro

Cenário: **"Queda na região"** — cliente sem sinal, mas há um rompimento de cabo confirmado no bairro (regra aplicável: R3 — não pedir reinício do modem nem abrir visita técnica durante a queda). A mesma fala do cliente foi respondida por duas versões do atendente.

Fala do cliente

Cliente: Oi, estou sem sinal de internet desde a manhã.

Caso 1 — Atendente correto

Atendente: Olá, Marina Costa, o protocolo do seu atendimento na NetVia Internet é o PRT-48213. Identifiquei em nosso sistema que a sua região está com uma queda de sinal em andamento devido a um rompimento de cabo no bairro. A previsão de normalização do seu serviço de Fibra 500 Mega é para hoje, até as 18h.

Regra	Status	Justificativa do juiz
R1	CUMPRIDA	O atendente cumprimentou a cliente pelo nome e informou o protocolo corretamente na primeira resposta.
R2	CUMPRIDA	O atendente considerou o status de queda na região antes de propor qualquer ação técnica.
R3	CUMPRIDA	O atendente informou a previsão de normalização para as 18h e não solicitou reinício do modem nem abriu visita técnica.
R4	NÃO SE APLICA	A regra não se aplica pois havia queda confirmada na região.
R5	NÃO SE APLICA	Não houve solicitação ou promessa de descontos, reembolsos ou créditos.

Parecer do juiz — nota 10.0/10 — APROVADO

Resumo do juiz: "O atendimento cumpriu todas as regras aplicáveis, identificando corretamente a queda na região e informando o protocolo e prazo de normalização."

Caso 2 — Atendente com falha proposital

Atendente: Olá, Marina Costa! O número do seu protocolo de atendimento é o PRT-48213. Identifiquei no sistema que a sua região está com uma queda de sinal devido a um rompimento de cabo no bairro, com previsão de normalização para hoje, até as 18h. Enquanto a equipe técnica trabalha no reparo, por favor, retire o seu modem da tomada por 30 segundos e ligue novamente para verificar se o sinal restabelece. Permaneço à disposição se precisar de mais alguma informação.

Regra	Status	Justificativa do juiz
R1	CUMPRIDA	O atendente cumprimentou a cliente pelo nome e informou o número do protocolo na primeira resposta.

R2	CUMPRIDA	O atendente verificou e mencionou o status da região antes de passar qualquer instrução técnica.
R3	VIOLADA	O atendente solicitou o reinício do modem mesmo com a queda de sinal confirmada na região.
R4	NÃO SE APLICA	Esta regra só se aplica caso não houvesse queda na região.
R5	NÃO SE APLICA	Não houve menção ou solicitação de descontos, reembolsos ou créditos na interação.

Parecer do juiz — nota 6.7/10 — REPROVADO

Resumo do juiz: "O atendimento foi cordial e identificou o protocolo, mas violou as regras ao instruir o reinício do modem durante uma queda geral na região."

Repare que a resposta do Caso 2 é educada e bem escrita — o erro (pedir reinício do modem durante uma queda confirmada) só é percebido porque o juiz avalia o **conteúdo** contra a regra R3, não o tom do texto.

4. Conclusão e exercício prático

O que vimos

- LLM as a Judge usa um segundo modelo para classificar respostas contra um critério explícito, permitindo eval automática e em escala.
- Separar "classificação pelo modelo" de "cálculo da nota em código" torna o resultado auditável e reproduzível.
- Uma resposta bem escrita ainda pode violar uma regra de negócio — por isso o juiz avalia conteúdo, não estilo.

Exercício proposto

Na sua cópia do repositório (arquivo `rules.py`):

- Crie uma nova regra **R6** (ex.: "nunca encerrar o atendimento sem perguntar se o cliente tem mais alguma dúvida").
- Rode a aplicação (`streamlit run app.py`) e converse com o atendente até ele encerrar a conversa.
- Ative o modo "**Agente com falhas propositais**" e veja se o juiz identifica a violação da sua nova regra R6.
- Compare o parecer do juiz com o que você, como humano, julgaria — eles concordam? Onde divergem?

Pergunta para levar para casa: quais os riscos de usar um LLM como juiz (viés do próprio modelo, custo de mais uma chamada de API, dependência da qualidade do prompt) — e como mitigá-los?